

bok25

**基
礎
班**

、考研数据結構

有向无环图DAG

拓扑排序

关键路径

有向无环图DAG

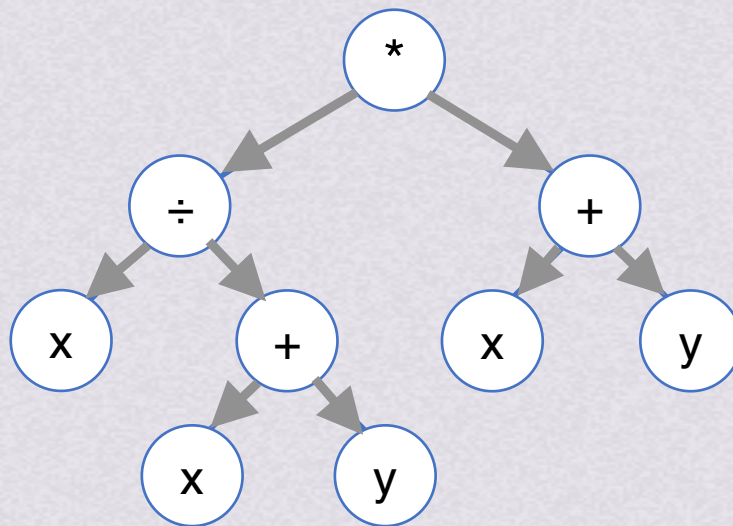
定义：不存在环的有向图，简称DAG图



有向无环图DAG

定义：不存在环的有向图，简称DAG图

表达式中的应用： $(x+y)*((x+y)/x)$



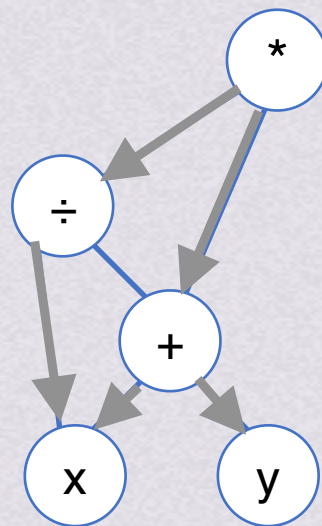
如何利用有向无环图将它化简？

- 1.不能出现重复的操作数顶点
- 2.尽可能化简

有向无环图DAG

定义：不存在环的有向图，简称DAG图

表达式中的应用： $(x+y)*((x+y)/x)$



- 1.不能出现重复的操作数顶点
- 2.尽可能化简

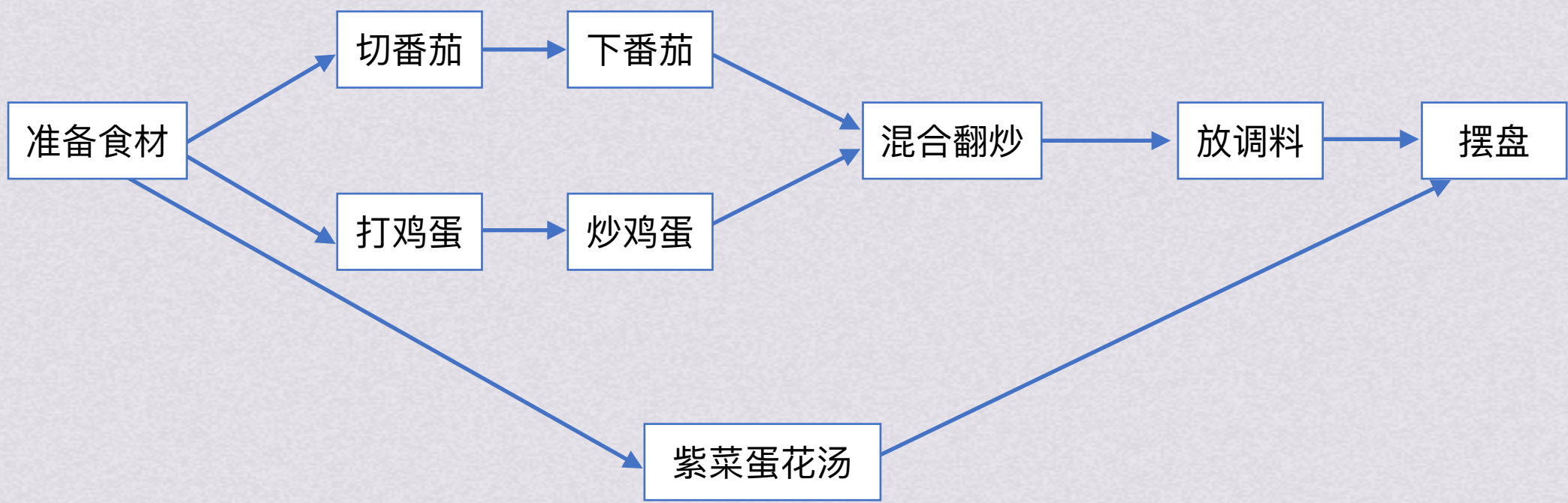
拓扑排序 AOV网

(Activity On Vertex Network)

用顶点表示活动，用弧表示活动间的优先关系，这样的有向图称为AOV网

若网中的所有顶点都在它的拓扑序列中，则该AOV网不存在环

拓扑排序：将AOV网中的所有顶点排成一个线性序列，且满足：
若在AOV网中从顶点 v_i 到 v_j 有一条路径，则该线性序列中的顶点 v_i 必定在顶点 v_j 之前
(在每个活动开始时，保证它的所有前驱活动都已完成)





拓扑排序 如何进行拓扑排序?

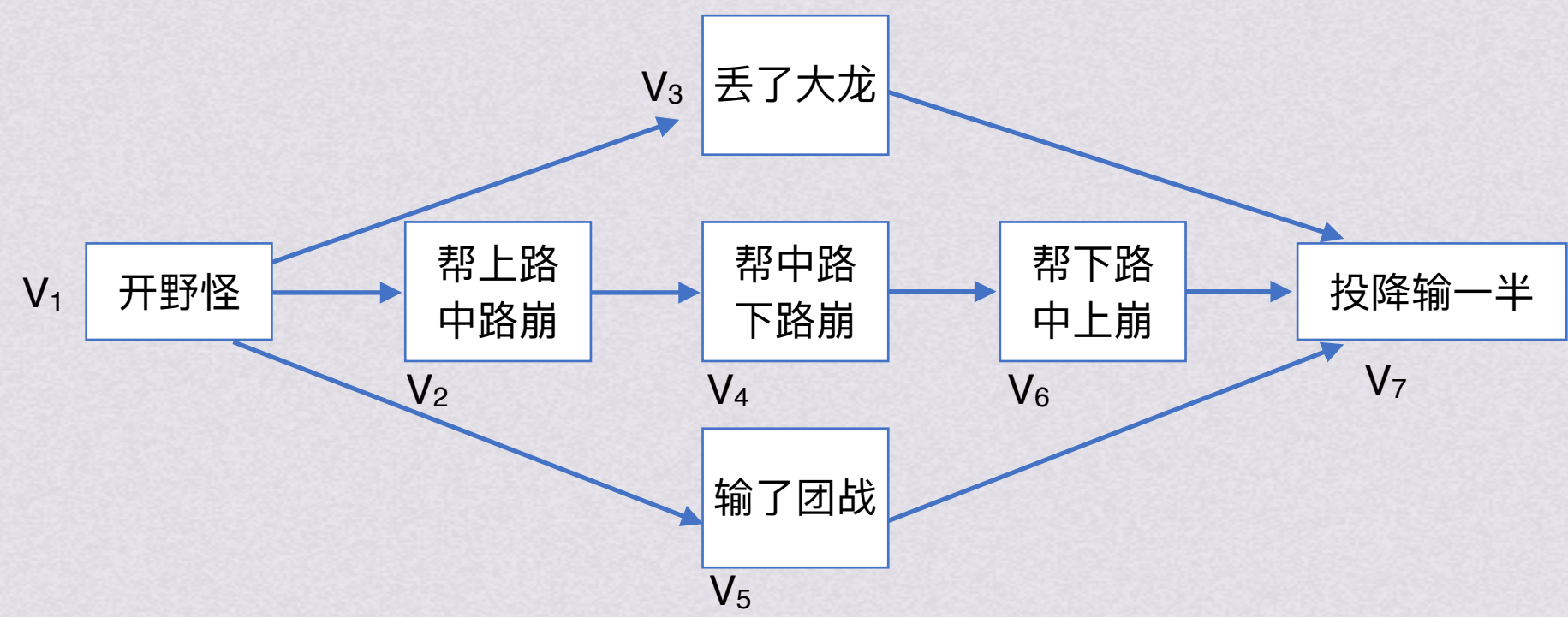
- 步骤:
- 1.在有向图中选择一个无前驱的顶点并输出它
 - 2.从图中删除该顶点和所有从它发射出去的弧

重复前两步操作，直到不存在无前驱的顶点

若此时输出的顶点数小于有向图中的顶点数，则图中存在环
否则输出的顶点序列即为该图的一个拓扑序列

拓扑排序 如何进行拓扑排序?

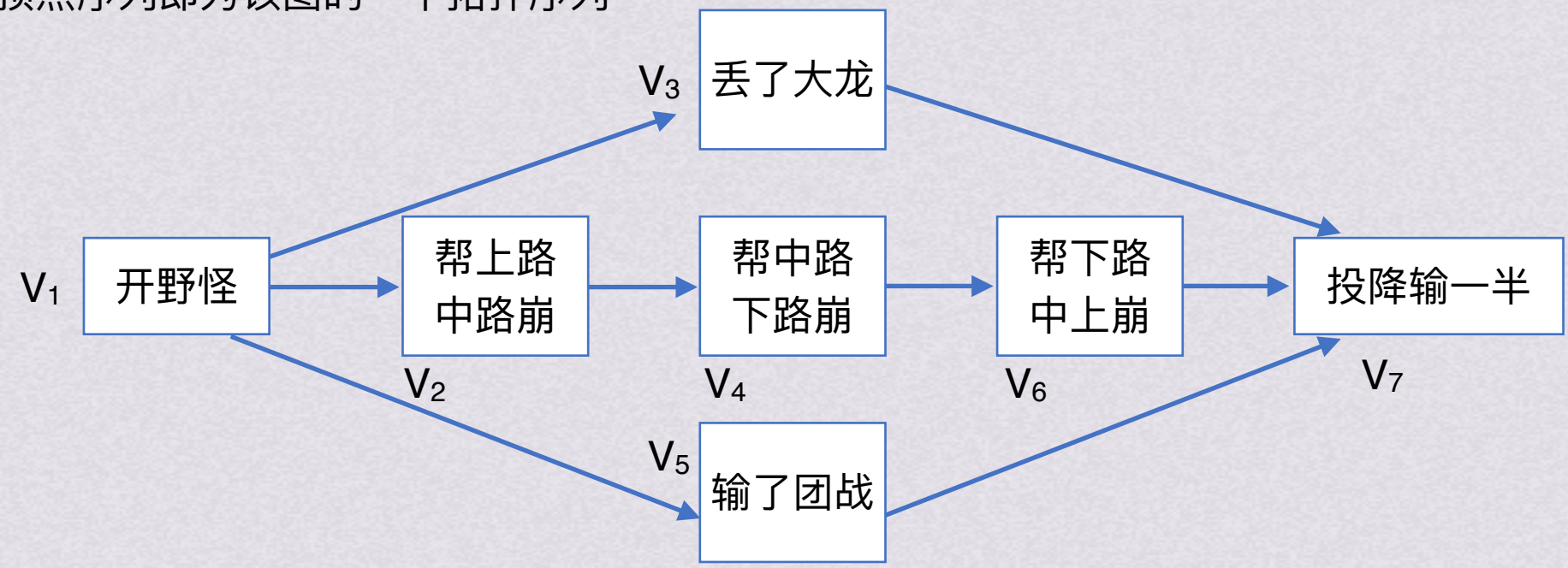
- 步骤:
- 1.在有向图中选择一个无前驱的顶点并输出它
 - 2.从图中删除该顶点和所有从它发射出去的弧
- 重复前两步操作，直到不存在无前驱的顶点
若此时输出的顶点数小于有向图中的顶点数，则图中存在环
否则输出的顶点序列即为该图的一个拓扑序列



拓扑排序 如何进行拓扑排序?

- 步骤:
- 1.在有向图中选择一个无前驱的顶点并输出它
 - 2.从图中删除该顶点和所有从它发射出去的弧

重复前两步操作，直到不存在无前驱的顶点
若此时输出的顶点数小于有向图中的顶点数，则图中存在环
否则输出的顶点序列即为该图的一个拓扑序列



拓扑排序输出序列: V₁ V₂ V₅ V₄ V₃ V₆ V₇
(其中一种)



拓扑排序 代码实现

```
// 访问标记数组，标记图中的顶点是否被访问过
bool visited[MVNUM];

// 对图G进行深度优先遍历
void DFSTraverse(Graph G)
{
    // 初始化所有顶点的访问标记为false（未访问）
    for(v=0; v<G.vexnum; ++v)
        visited[v]=false;
    // 时间戳初始化为0
    time=0;
    // 遍历图中的每个顶点
    for(v=0; v<G.vexnum; ++v)
        // 如果该顶点未被访问，从该顶点开始进行深度优先搜索
        if(!visited[v]) DFS(G,v);
}

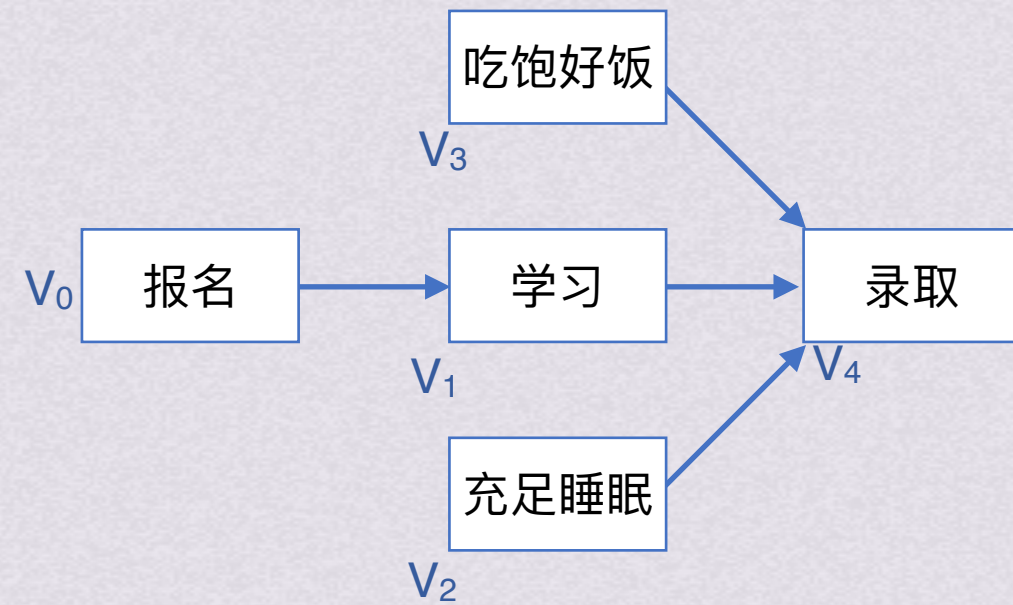
// 从顶点v开始进行深度优先搜索
void DFS(Graph G, int v)
{
    // 标记当前顶点v为已访问
    visited[v]=TRUE;
    // 访问当前顶点v
    visit(v);
    // 遍历v的所有邻接顶点
    for(w=FirstNeighbor(G,v); w>=0; w=NextNeighbor(G,v,w))
        // 如果邻接顶点未被访问，递归地对其进行深度优先搜索
        if(!visited[w]) DFS(G,w);
    // 遍历完成后，增加时间戳，并记录顶点v的完成时间
    time=time+1;
    finishTime[v]=time;
}
```

拓扑排序 代码实现

```
// 访问标记数组，标记图中的顶点是否被访问过
bool visited[MVNUM];

// 对图G进行深度优先遍历
void DFSTraverse(Graph G)    传入图G
{
    // 初始化所有顶点的访问标记为false（未访问）
    for(v=0; v<G.vexnum; ++v)
        visited[v]=false;
    // 时间戳初始化为0
    time=0;
    // 遍历图中的每个顶点
    for(v=0; v<G.vexnum; ++v) v=4
        // 如果该顶点未被访问，从该顶点开始进行深度优先搜索
        if(!visited[v]) DFS(G,v); visited[4]=1
}

// 从顶点v开始进行深度优先搜索
void DFS(Graph G, int v)
{
    // 标记当前顶点v为已访问
    visited[v]=TRUE;
    // 访问当前顶点v
    visit(v);
    // 遍历v的所有邻接顶点
    for(w=FirstNeighbor(G,v); w>=0; w=NextNeighbor(G,v,w))
        // 如果邻接顶点未被访问，递归地对其进行深度优先搜索
        if(!visited[w]) DFS(G,w);
    // 遍历完成后，增加时间戳，并记录顶点v的完成时间
    time=time+1;
    finishTime[v]=time;
}
time=5
```



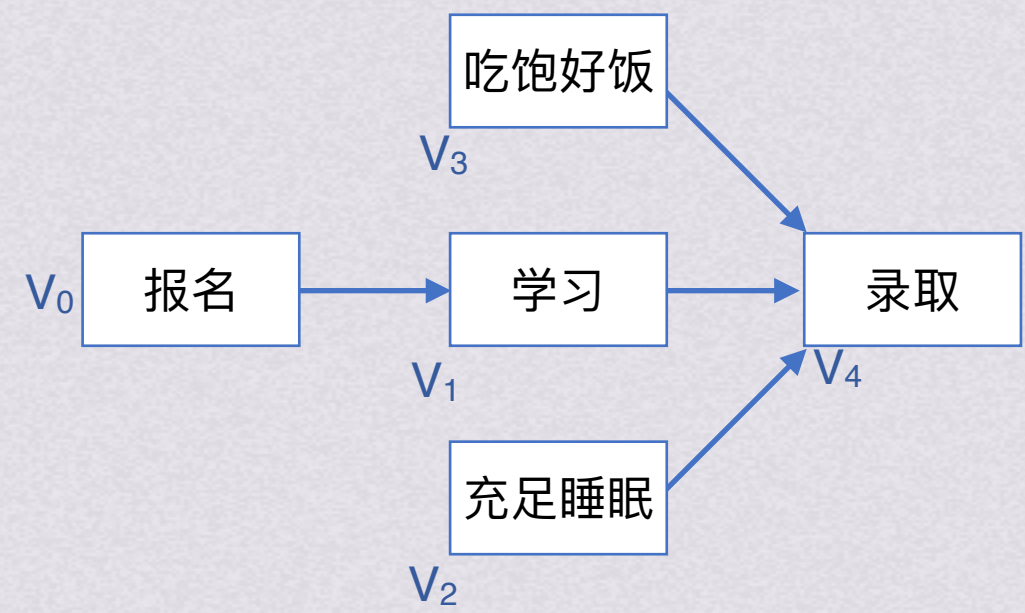
图G

递归栈

finishtime[5]	3	2	4	5	1
	0	1	2	3	4
visited[5]	1	1	1	1	1
	0	1	2	3	4

拓扑排序 代码实现

finishtime[5]	3	2	4	5	1
	0	1	2	3	4



图G



关键路径

AOV网

用顶点表示活动，用弧表示活动间的优先关系，这样的有向图称为AOV网

AOE网

用顶点表示事件，而弧表示活动，弧上的权值表示完成该活动的开销



关键路径

AOE网

用顶点表示事件，而弧表示活动，弧上的权值表示完成该活动的开销

AOE网常用于规划工程，由于整个工程只有一个开始点和完成点，所以在正常情况下，网中只会有一个入度为0的点，称为源点。

也只有一个出度为0的点，称为汇点

从源点到汇点的带权路径长度最长的路径，称为关键路径

关键路径上的活动（弧）称为**关键活动**，这些活动是影响工程进度的关键，它们的提前和延期将使整个工程提前或延期

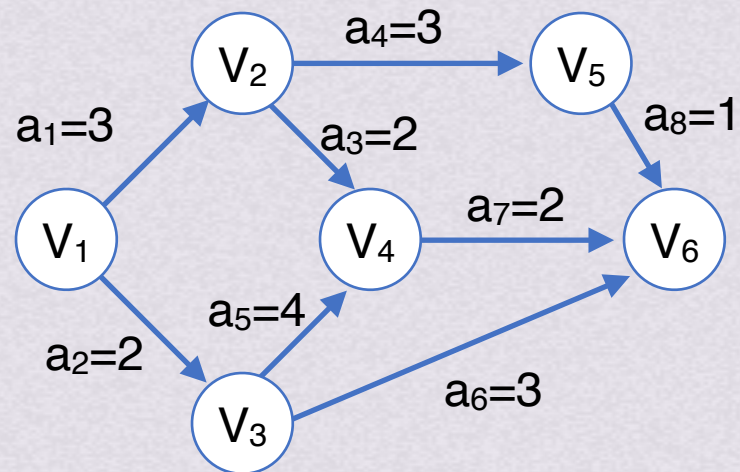
关键路径

事件 V_i 的最早发生时间 V_e

事件 V_i 的最迟发生时间 V_l

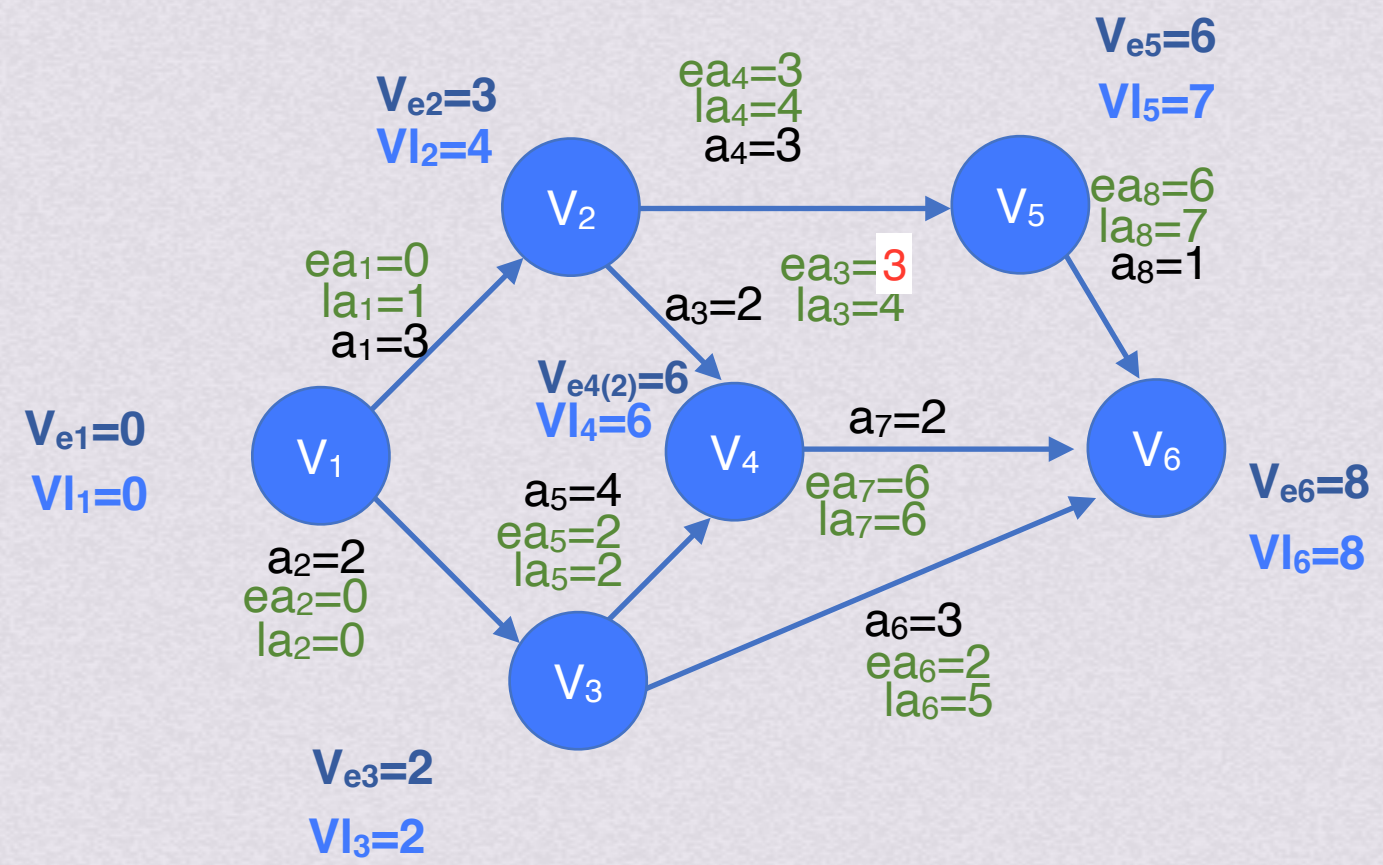
活动 a_i 的最早开始时间 e

活动 a_i 的最迟开始时间 l



关键路径

- 事件 V_i 的最早发生时间 V_e
- 事件 V_i 的最迟发生时间 V_l
- 活动 a_i 的最早开始时间 e
- 活动 a_i 的最迟开始时间 l



一个事件要想发生，那么它前面的活动必须全部结束才可以

每个顶点（事件）的最迟开始时间要根据顺序从汇点开始倒推

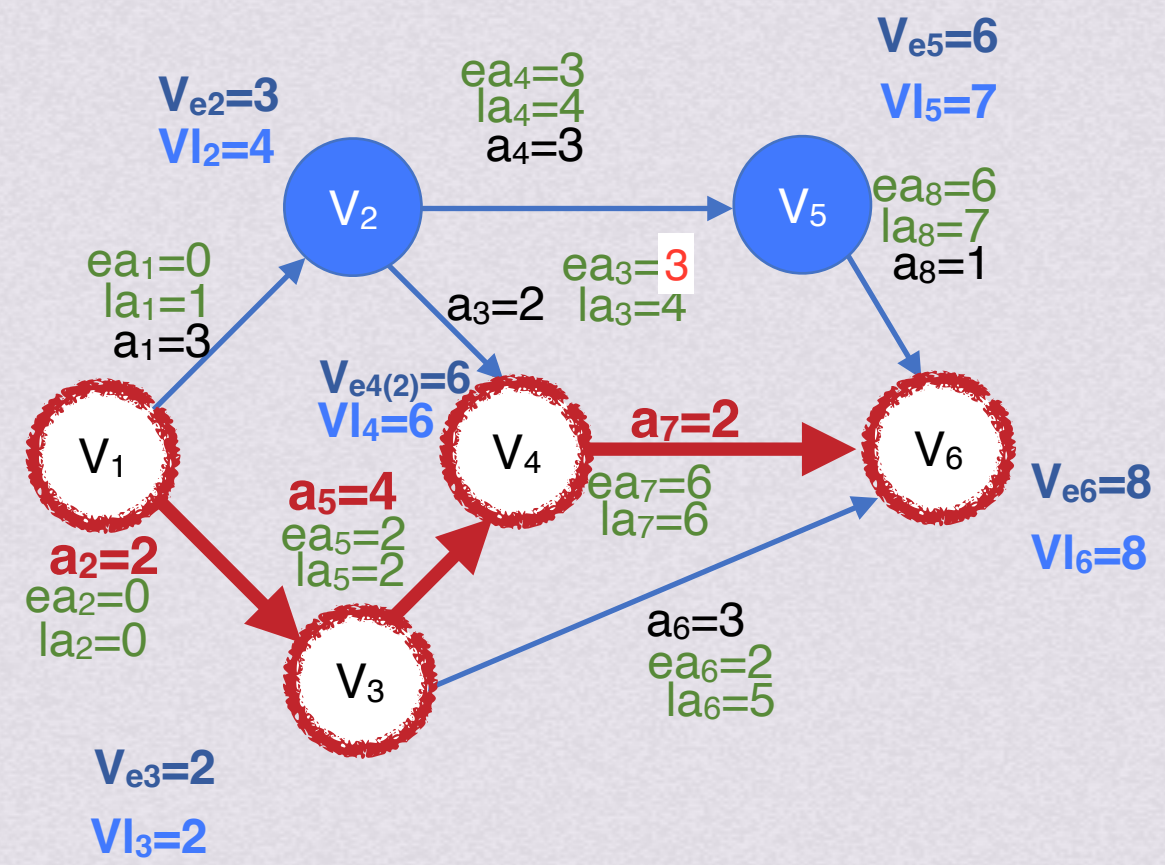
弧（活动）的最早开始时间 e 等于该弧起点顶点的 v_e

弧（活动）的最迟开始时间 l 等于该弧终点的顶点的 v_l 减去该弧持续的时间

关键路径

事件 V_i 的最早发生时间 V_e
事件 V_i 的最迟发生时间 V_l
活动 a_i 的最早开始时间 e
活动 a_i 的最迟开始时间 l

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
$e(i)$	0	0	3	3	2	2	6	6
$l(i)$	1	0	4	4	2	5	6	7
$l(i)-e(i)$	1	0	1	1	0	3	0	1



一个事件要想发生，那么它前面的活动必须全部结束才可以

每个顶点（事件）的最迟开始时间要根据顺序从汇点开始倒推

弧（活动）的最早开始时间 e 等于该弧起点顶点的 v_e

弧（活动）的最迟开始时间 l 等于该弧终点的顶点的 v_l 减去该弧持续的时间



关键路径 注意事项

1. 关键路径并不唯一
2. 关键路径是权值之和最大的那条路径
3. 增加关键路径上的任一关键活动的持续时间，一定会延长整个工期
4. 缩短关键路径，并不一定导致整个工期加快
 - 如果只有一条关键路径，缩短关键路径，一定导致整个工期加快
 - 如果有多条关键路径，缩短关键路径，不一定导致整个工期加快
因为缩短了路径长度的那条关键路径可能不再是关键路径